

Московский Авиационный Институт
(национальный исследовательский университет)

Институт 8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1

по дисциплине «Численные методы»

Студент: Филиппов В.М.

Группа: М8О-310Б-22

Преподаватель: Шабунина А.А.

Дата:

Оценка:

Подпись:

Москва, 2025

Содержание

LU-разложение.....	3
Метод прогонки.....	10
Метод простых итераций.....	14
Метод вращения Якоби.....	18
QR-алгоритм.....	26

LU-разложение

Теория

Одним из ключевых инструментов численного решения линейных систем является LU-разложение. Оно представляет исходную матрицу A в виде произведения двух треугольных матриц:

$$A=L \cdot U,$$

где:

- L — нижняя треугольная матрица (все элементы выше главной диагонали равны нулю),
- U — верхняя треугольная матрица (все элементы ниже главной диагонали равны нулю).

Это разложение тесно связано с методом Гаусса: матрица U соответствует ступенчатому виду, полученному в процессе исключения переменных, а матрица L хранит коэффициенты, использованные для преобразования строк.

При программной реализации важно учитывать вычислительную устойчивость. Например, деление на малые числа может привести к значительным ошибкам. Для минимизации таких эффектов применяется частичный выбор ведущего элемента: на каждом шаге выбирается максимальный по модулю элемент в столбце, и соответствующая строка перемещается на текущую позицию. Это требует введения дополнительной матрицы перестановок P , которая фиксирует все выполненные перестановки строк.

Матрица P не только обеспечивает устойчивость вычислений, но и позволяет:

1. Корректно преобразовывать правую часть системы при перестановках.
2. Находить обратную матрицу A^{-1} для исходной системы, а не для модифицированной версии с переставленными строками.

Использование LU-разложения особенно выгодно при необходимости многократного решения систем с одной и той же матрицей A , но разными правыми частями. После однократного построения L и U каждое новое решение требует лишь обратной подстановки, что существенно экономит вычислительные ресурсы.

Код программы

Ниже представлен код программы:

```
#pragma once
#include <iostream>
#include <vector>
#include <cmath>
#include "matrix.h"
#include <iomanip>
#include <stdexcept>
```

```

class LU {

public:
    LU() = default;

    template <class T>
    typename std::enable_if<std::is_arithmetic<T>::value, std::pair<Matrix<double>,
Matrix<double>>>>::type
    decompose(Matrix<T>& A, bool info = 0)
    {
        Matrix<double> L(A.getRows(), A.getRows(), 0.), U(A.getRows(), A.getRows(), 0.);
        std::vector<int> P(A.getRows());
        for (int i = 0; i < A.getRows(); ++i) {
            P[i] = i;
        }
        int permutatuions = 0;

        for (int k = 0; k < A.getRows(); ++k) {
            double max_elem = abs(A.at(k, k));
            int max_ind = k;
            for (int i = k + 1; i < A.getRows(); ++i) {
                if (abs(A.at(i, k)) > max_elem) {
                    max_elem = abs(A.at(i, k));
                    max_ind = i;
                }
            }

            if (max_elem == 0) {
                throw std::invalid_argument("Broken matrix!");
            }

            if (max_ind != k) {
                ++permutatuions;
                A.swapRows(k, max_ind);
            }
        }
    }
};

```

```

        std::swap(P[k], P[max_ind]);
    }
}

for (int k = 0; k < A.getRows(); ++k) {
    U.at(k, k) = A.at(k, k);
    L.at(k, k) = 1;
    for (int i = k + 1; i < A.getRows(); ++i) {
        L.at(i, k) = A.at(i, k) / U.at(k, k);
        U.at(k, i) = A.at(k, i);
    }

    for (int i = k + 1; i < A.getRows(); ++i) {
        for (int j = k + 1; j < A.getRows(); ++j) {
            A.at(i, j) -= L.at(i, k) * U.at(k, j);
        }
    }
}

if (info) {
    std::cout << "determinant: " << determinant(L, U) * (permutatuions % 2 == 0 ? 1 : -1) <<
'\n';

    std::cout << "inverted\n";
    Matrix<double> AInv = invert(L, U, P);
    AInv.print();
}

return std::make_pair(L, U);
}

template <class T>
typename std::enable_if<std::is_arithmetic<T>::value, std::vector<double>>::type
solve(Matrix<T> A)
{
    auto [L, U] = decompose(A);

    std::vector<double> x;

```

```

    for (int i = 0; i < A.getRows(); ++i) {
        x.push_back(A.at(i, A.getCols() - 1));
    }

    return backward_substitution(U, forward_substitution(L, x));
}

private:
    std::vector<double> forward_substitution(const Matrix<double>& L, const
std::vector<double>& b)
    {
        size_t n = L.getRows();
        std::vector<double> y(n, 0);

        for (size_t i = 0; i < n; ++i) {
            y[i] = b[i];
            for (size_t j = 0; j < i; ++j) {
                y[i] -= L.at(i, j) * y[j];
            }
        }
        return y;
    }

    std::vector<double> backward_substitution(const Matrix<double>& U, const
std::vector<double>& y)
    {
        size_t n = U.getRows();
        std::vector<double> x(n, 0);

        for (int i = n - 1; i >= 0; --i) {
            x[i] = y[i];
            for (size_t j = i + 1; j < n; ++j) {
                x[i] -= U.at(i, j) * x[j];
            }
            x[i] /= U.at(i, i);
        }
    }

```

```

    }

    return x;
}

double determinant(const Matrix<double>& L, const Matrix<double>& U)
{
    double det = 1;
    for (int i = 0; i < U.getRows(); ++i) {
        det *= (U.at(i, i) * L.at(i, i));
    }
    return det;
}

Matrix<double> invert(const Matrix<double>& L, const Matrix<double>& U, const
std::vector<int> P)
{
    Matrix<double> invA(L.getRows(), L.getRows());

    for (int i = 0; i < L.getRows(); i++) {
        std::vector<double> e(L.getRows(), 0.0);
        e[i] = 1.0;

        std::vector<double> Y = forward_substitution(L, e);
        std::vector<double> X = backward_substitution(U, Y);

        for (int j = 0; j < L.getRows(); j++)
            invA.at(j, P[i]) = X[j];
    }
    return invA;
}

};

#include "lu.h"
#include "matrix.h"
#include <fstream>

```

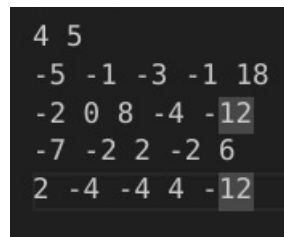
```

int main() {
    LU lu;
    Matrix<double>
mat("/home/zloyaloha/programming/Numerical-analysis/lab1/lu/test/test.txt");
    std::vector<double> res = lu.solve(mat);
    for (auto& x: res) {
        std::cout << x << ' ';
    }
    std::cout << std::endl;
    mat.checkEquality(res);
    lu.decompose(mat, 1);
}

```

Формат входных данных

На рисунке 1.1 приведен пример входных данных.



```

4 5
-5 -1 -3 -1 18
-2 0 8 -4 -12
-7 -2 2 -2 6
2 -4 -4 4 -12

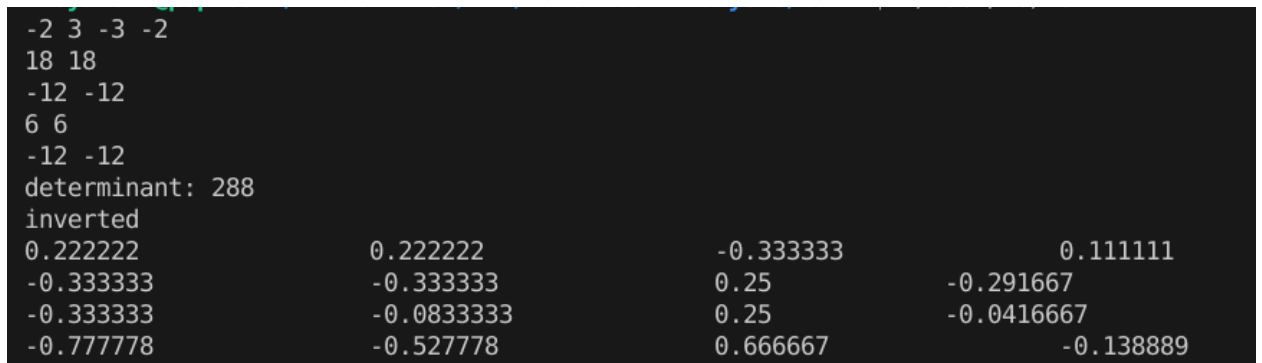
```

Рисунок 1.1 – Пример входных данных

На первой строке вводятся два числа n , m – количество уравнений в системе и количество столбцов в матрице системы (количество неизвестных + свободный член). На следующих n строках вводятся коэффициенты матрицы системы.

Формат выходных данных

На рисунке 1.2 приведены выходные данные, являющиеся решением задачи с рисунка 1.1.



```

-2 3 -3 -2
18 18
-12 -12
6 6
-12 -12
determinant: 288
inverted
0.222222      0.222222      -0.333333      0.111111
-0.333333     -0.333333      0.25      -0.291667
-0.333333     -0.0833333     0.25      -0.0416667
-0.777778     -0.527778      0.666667     -0.138889

```

Рисунок 1.2 – Решение задачи

Сначала выводятся корни системы в строку. Далее идет проверка совпадения свободных членов и подставления корней в исходную систему. Потом выводится определитель матрицы системы. Затем выводится обратная матрица.

Заключение

LU-разложение - это мощный инструмент для работы с системами линейных уравнений, позволяющий значительно оптимизировать вычисления. Его основная идея заключается в представлении исходной матрицы системы в виде произведения двух треугольных матриц (нижней L и верхней U), что открывает широкие возможности для последующих вычислений.

Главное преимущество такого подхода в том, что после однократного выполнения разложения (что требует порядка $O(n^3)$ операций) мы получаем возможность решать множество задач с этой матрицей уже за квадратичное время $O(n^2)$.

Такая эффективность делает LU-разложение универсальным и экономичным методом, особенно полезным в случаях, когда нужно многократно работать с одной и той же матрицей системы, но с разными правыми частями или когда требуется комплексный анализ матричных свойств.

Метод прогонки

Теория

Метод прогонки - это оптимизированный вариант метода Гаусса для систем с трехдиагональными матрицами, требующий всего $O(n)$ операций вместо $O(n^3)$. Трехдиагональная матрица имеет вид, где все элементы заполнены нулями, кроме трех диагоналей: главной, над и под ней.

На рисунке ниже показан вид системы, матрица которой является трехдиагональной.

$$a_1 = 0 \left\{ \begin{array}{l} b_1 x_1 + c_1 x_2 = d_1 \\ a_2 x_1 + b_2 x_2 + c_2 x_3 = d_2 \\ a_3 x_2 + b_3 x_3 + c_3 x_4 = d_3 \\ \dots \\ a_{n-1} x_{n-2} + b_{n-1} x_{n-1} + c_{n-1} x_n = d_{n-1} \\ a_n x_{n-1} + b_n x_n = d_n, \quad c_n = 0, \end{array} \right.$$

Решение такой системы будет в виде:

$$x_i = P_i x_{i+1} + Q_i$$

Коэффициенты же в свою очередь будут находиться по формулам:

$$P_i = \frac{-c_i}{b_i + a_i P_{i-1}}$$

$$Q_i = \frac{d_i - a_i Q_{i-1}}{b_i + a_i P_{i-1}}$$

Таким образом, сначала прямым прогоном осуществляем поиск всех коэффициентов, а обратным – поиск самих значений неизвестных.

Код программы

Ниже представлен код программы:

```
#include <iostream>
#include "matrix.h"
#include <vector>
#include <stdexcept>

class Tridiagonal {
public:
    Tridiagonal() = default;

    template <class T>
    typename std::enable_if<std::is_arithmetic<T>::value, std::vector<double>>::type
```

```

solve(Matrix<T>& A)
{
    int n = A.getRows();
    double a = 0, b = A.at(0, 0), c = A.at(0, 1), d = A.at(0, A.getCols() - 1);
    std::vector<double> P(n), Q(n);
    P[n - 1] = 0;
    P[0] = -c / b; Q[0] = d / b;
    int counter = 0;
    for (int i = 1; i < n; ++i) {
        a = A.at(i, i - 1);
        b = A.at(i, i);
        c = A.at(i, i + 1);
        d = A.at(i, A.getCols() - 1);

        if (a == 0 || c == 0) {
            std::cerr << "the method may not converge. a or c is zero" << std::endl;
        }
        if (b < a + c) {
            std::cerr << "the method may not converge. b is lower then a + c" << std::endl;
        }
        if (b > a + c) {
            ++counter;
        }

        P[i] = -c / (b + a * P[i - 1]);
        Q[i] = (d - a * Q[i - 1]) / (b + a * P[i - 1]);
    }
    if (counter == 0) {
        std::cerr << "the method may not converge. b is not strongly bigger then a + c at least
once" << std::endl;
    }
    std::vector<double> x(n);
    x[n - 1] = Q[n - 1];
    for (int i = n - 2; i >= 0; --i) {
        x[i] = P[i] * x[i + 1] + Q[i];
    }
}

```

```

    }
    return x;
}
};

#include "tridiagonal-matrix.h"
#include <vector>

int main() {
    Tridiagonal tr;
    Matrix<double>
mat("/home/zloyaloha/Yandex.Disk/MAI/Numerical-analysis/lab1/tridiagonal-matrix/
test/test.txt");
    std::vector<double> res = tr.solve(mat);
    for (auto& x: res) {
        std::cout << x << ' ';
    }
    std::cout << std::endl;

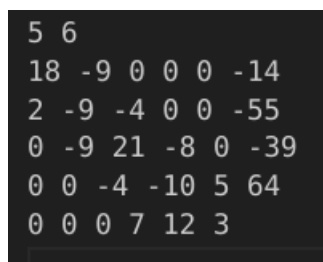
    mat.checkEquality(res);

    // lu.decompose(mat, 1);
}

```

Формат входных данных

На рисунке 2.1 приведен пример входных данных.



```

5 6
18 -9 0 0 0 -14
2 -9 -4 0 0 -55
0 -9 21 -8 0 -39
0 0 -4 -10 5 64
0 0 0 7 12 3

```

Рисунок 2.1 – Пример входных данных

На первой строке вводятся два числа n , m – количество уравнений в системе и количество столбцов в матрице системы (количество неизвестных + свободный член). На следующих n строках вводятся коэффициенты матрицы системы.

Формат выходных данных

На рисунке 2.2 приведены выходные данные, являющиеся решением задачи с рисунка 2.1.

```
the method may not converge. b is lower then a + c
the method may not converge. b is lower then a + c
2.72102 6.99759 -0.634065 -4.66171 2.96933
-14 -14
-55 -55
-39 -39
64 64
3 3
```

Рисунок 2.2 – Решение задачи

Выводятся предупреждения о том, что метод может не сойтись, корни уравнения, а также выполняется проверка.

Заключение

Метод прогонки обладает существенными преимуществами по сравнению с более универсальными методами вроде LU-разложения, когда речь идет о системах с трехдиагональными матрицами. Его ключевое достоинство заключается в исключительной вычислительной эффективности - алгоритм требует всего $O(n)$ операций благодаря двум последовательным проходам: прямому для вычисления прогоночных коэффициентов и обратному для нахождения решения. Это кардинально отличается от $O(n^3)$ операций, необходимых для LU-разложения.

Простота реализации метода прогонки делает его особенно привлекательным - достаточно двух компактных циклов вместо сложных матричных преобразований. Однако важно понимать, что область применения этого метода ограничена системами с трехдиагональной структурой. Когда такая специальная структура матрицы имеется или может быть достигнута путем преобразований, выбор в пользу метода прогонки становится очевидным с точки зрения как производительности, так и простоты кодирования. В таких случаях его использование предпочтительнее, чем применение более ресурсоемких общих методов решения линейных систем.

Метод простых итераций

Теория

При решении систем линейных алгебраических уравнений большой размерности прямые методы (за исключением метода прогонки) сталкиваются с существенными вычислительными трудностями, связанными с хранением и обработкой матриц. Особенно это касается разреженных матриц, где количество ненулевых элементов значительно меньше общего числа элементов. В таких случаях наиболее эффективными оказываются итерационные методы, основанные на последовательном приближении к решению.

Основная идея итерационных подходов заключается в преобразовании исходной системы $Ax = b$ к виду $x = Cx + d$, где матрица C и вектор d получаются путем выражения каждой переменной через остальные компоненты. Итерационный процесс начинается с произвольного начального приближения, обычно принимаемого равным вектору правых частей или нулевому вектору, и продолжается до достижения заданной точности, определяемой условием $\|x^{(k+1)} - x^{(k)}\| < \epsilon$.

Критерием сходимости такого процесса служит условие $\|C\| < 1$, где норма матрицы вычисляется как максимальная сумма модулей элементов строки. Хотя метод простых итераций отличается простотой реализации, его сходимость часто оказывается недостаточно быстрой. Более эффективным вариантом является метод Зейделя, который модифицирует процесс вычислений за счет использования уже обновленных компонент решения на текущей итерации при расчете последующих переменных.

Метод Зейделя, сохраняя общую схему преобразования системы, ускоряет сходимость за счет более полного использования текущей информации. Как и в методе простых итераций, процесс начинается с вектора правых частей, но на каждом шаге учитываются уже вычисленные на данной итерации значения, что приводит к более быстрому достижению требуемой точности решения. Такой подход особенно эффективен для больших разреженных систем, где прямые методы становятся непрактичными из-за вычислительных затрат.

Код программы

Ниже представлен код программы метода простых итераций:

```
#include <iostream>
#include "matrix.h"
#include <vector>
#include <stdexcept>
#include <cmath>
```

```

const double EPS = 1e-6; // Точность вычислений
const double max_iter = 10; // Точность вычислений

class Zeydel {
public:
    Zeydel() = default;

    template <class T>
    typename std::enable_if<std::is_arithmetic<T>::value, std::vector<double>>::type
    solve(Matrix<T>& A)
    {
        int k = 0;
        double max_norm = 0;
        std::vector<double> x(A.getRows(), 0), x_prev(A.getRows(), 0);
        Matrix<double> A_(A.getRows(), A.getCols(), 0.);
        for (int i = 0; i < A.getRows(); ++i) {
            while (A.at(i, i) == 0) {
                ++k;
                A.swapRows(i, i + k);
            }
            A_.at(i, 0) = A.at(i, A.getCols() - 1) / A.at(i, i); // свободный член на последнем месте
            k = 0;
            for (int j = 0; j < A.getCols() - 1; ++j) {
                A_.at(i, j + 1) = A.at(i, j) / A.at(i, i); // тут все как обычно
            }
            A_.at(i, i + 1) = 0;
            x[i] = A_.at(i, 0);
        }

        for (int iter = 0; iter < max_iter; ++iter) {
            x_prev = x; max_norm = 0;
            for (int i = 0; i < A.getRows(); ++i) {
                double sum = A_.at(i, 0);
                for (int j = 0; j < A.getCols() - 1; ++j) {

```

```

        sum -= A_.at(i, j + 1) * x[j];
    }
    x[i] = sum;
    max_norm = std::max(max_norm, fabs(x[i] - x_prev[i]));
}
if (max_norm < EPS) {
    std::cout << "Needed iterations: " << iter << std::endl;
    return x;
}
}
};

#include "zeydel.h"
#include <vector>

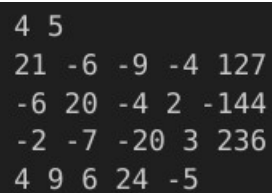
int main() {
    Zeydel tr;
    Matrix<double> mat("/home/zloyaloha/programming/Numerical-analysis/lab1/zeydel/test/
test.txt");
    std::vector<double> res = tr.solve(mat);
    for (auto& x: res) {
        std::cout << x << ' ';
    }
    std::cout << std::endl;

    mat.checkEquality(res);
}

```

Формат входных данных

На рисунке 3.1 приведен пример входных данных.



```

4 5
21 -6 -9 -4 127
-6 20 -4 2 -144
-2 -7 -20 3 236
4 9 6 24 -5

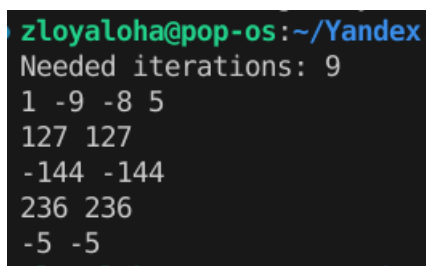
```

Рисунок 3.1 – Пример входных данных

На первой строке вводятся два числа n , m – количество уравнений в системе и количество столбцов в матрице системы (количество неизвестных + свободный член). На следующих n строках вводятся коэффициенты матрицы системы. В консоли вводится задаваемая точность.

Формат выходных данных

На рисунке 3.2 приведены выходные данные, являющиеся решением задачи с рисунка 3.1.



```
zloyaloha@pop-os:~/Yandex.  
Needed iterations: 9  
1 -9 -8 5  
127 127  
-144 -144  
236 236  
-5 -5
```

Рисунок 3.2 – Решение задачи

Выводится количество итераций, которые необходимо было произвести, чтобы ответ сошелся с достаточной точностью, затем выводятся корни в одной строке, а потом выводится проверка (подставляются в уравнение найденные корни).

Заключение

Метод простых итераций представляет собой достаточно простой и удобный в реализации подход к решению систем линейных уравнений, особенно по сравнению с более сложными методами вроде LU-разложения. Его главное преимущество заключается в простоте программирования - для реализации требуется всего несколько строк кода. Однако важно понимать, что данный метод имеет ограниченную область применения и сходится не для всех систем уравнений. В моем случае достаточное условие сходимости выполнялось, что позволило успешно найти решение без каких-либо осложнений.

Более эффективной модификацией этого метода является метод Зейделя, который отличается тем, что при вычислении каждого следующего приближения использует уже обновленные значения переменных. Такой подход обеспечивает существенное ускорение сходимости - в моих расчетах при одинаковой заданной точности метод Зейделя показал двукратное преимущество по скорости сходимости по сравнению с методом простых итераций. Это делает его особенно полезным при работе с большими системами уравнений, где скорость получения решения имеет критическое значение.

Метод вращения Якоби

Теория

Метод вращений Якоби представляет собой эффективный итерационный алгоритм для решения полной проблемы собственных значений симметрических матриц. Этот метод позволяет одновременно находить все собственные значения и соответствующие им собственные векторы с заданной точностью ϵ . Алгоритм основан на последовательном применении ортогональных преобразований, приводящих исходную матрицу к диагональному виду.

Основная идея метода заключается в использовании специальных ортогональных преобразований - вращений Якоби, которые на каждом шаге обнуляют максимальный по модулю внедиагональный элемент матрицы. Процесс начинается с симметрической матрицы A и заключается в построении последовательности матриц A_k , сходящейся к диагональной матрице Λ . При этом собственные значения исходной матрицы оказываются расположенными на главной диагонали Λ , а собственные векторы формируются как столбцы произведения всех примененных матриц вращения.

Метод особенно эффективен для симметрических матриц, где гарантируется сходимость к диагональной форме. Главное преимущество алгоритма - его устойчивость и возможность одновременного получения как собственных значений, так и собственных векторов. Точность решения контролируется параметром ϵ , который определяет критерий остановки итерационного процесса.

Код программы

```
#include <iostream>
#include "matrix.h"
#include <vector>
#include <stdexcept>
#include <tgmath.h>
#include <math.h>

const double EPSILON = 0.001;

struct MatrixElem {
    int row;
    int col;
    long double value;
};
```

```

class Rotating {
public:
    Rotating(long double epsilon) : eps(epsilon) {}

    std::pair<std::vector<long double>, Matrix<long double>> solve(Matrix<long double>
matrix);
private:
    MatrixElem findMaxOverDiagonal(const Matrix<long double>& matrix);
    long double calculateNonDiagonalSummary(const Matrix<long double>& matrix);
    long double eps;
};

std::pair<std::vector<long double>, Matrix<long double>> Rotating::solve(Matrix<long double>
matrix)
{
    std::vector<Matrix<long double>> Us;
    int k = 0;

    while(calculateNonDiagonalSummary(matrix) > eps) {
        MatrixElem max_elem = findMaxOverDiagonal(matrix);
        long double phi, cos_phi, sin_phi;
        if (std::fabs(matrix.at(max_elem.col, max_elem.col) - matrix.at(max_elem.row,
max_elem.row)) < eps) {
            cos_phi = 1 / sqrt(2);
            sin_phi = 1 / sqrt(2);
        } else {
            long double theta = (matrix.at(max_elem.row, max_elem.row) - matrix.at(max_elem.col,
max_elem.col)) / (2.0 * matrix.at(max_elem.col, max_elem.row));
            if (std::fabs(theta) < 1e-6) {
                theta = copysign(1e-6, theta); // Минимальный угол, чтобы не зависнуть
            }
            long double t = copysign(1.0 / (fabs(theta) + sqrt(1.0 + theta * theta)), theta);
            cos_phi = 1 / sqrt(1 + t * t);
            sin_phi = t * cos_phi;
        }
    }
}

```

```

    }

    Matrix<long double> U(matrix.getCols(), matrix.getRows(), 0);
    for (int i = 0; i < matrix.getRows(); ++i) {
        U.at(i, i) = 1;
    }

    Us.push_back(U);

    Us[k].at(max_elem.row, max_elem.row) = cos_phi; Us[k].at(max_elem.row, max_elem.col)
= -sin_phi;
    Us[k].at(max_elem.col, max_elem.row) = sin_phi; Us[k].at(max_elem.col, max_elem.col)
= cos_phi;
    matrix = ~Us[k] * matrix * Us[k];

    if (k > 100 * matrix.getRows()) {
        break;
    }
    ++k;
}

if (calculateNonDiagonalSummary(matrix) < eps) {
    std::cout << "Result for accuracy = " << eps << " by " << k << " iterations" << std::endl;
} else {
    std::cout << "Can't reach accuracy = " << eps << " by " << k << " iterations" << std::endl;
}

std::cout << "eigenvalues \n\n";
std::vector<long double> eigenvalues;
for (int i = 0; i < matrix.getCols(); ++i) {
    eigenvalues.push_back(matrix.at(i, i));
    std::cout << eigenvalues[i] << std::endl;
}
Matrix<long double> eigenvectors = Us[0] * Us[1];

```

```

    for (int i = 2; i < Us.size(); ++i) {
        eigenvectors = eigenvectors * Us[i];
    }

    std::cout << "\neigenvectors \n\n";
    eigenvectors.print();

    return std::make_pair(eigenvalues, eigenvectors);
}

MatrixElem Rotating::findMaxOverDiagonal(const Matrix<long double>& matrix) {
    MatrixElem max_elem;
    max_elem.value = 0;
    for (int i = 0; i < matrix.getCols(); ++i) {
        for (int j = i + 1; j < matrix.getRows(); ++j) {
            if (max_elem.value < std::fabs(matrix.at(i, j))) {
                max_elem.value = std::fabs(matrix.at(i, j));
                max_elem.row = i;
                max_elem.col = j;
            }
        }
    }
    return max_elem;
}

long double Rotating::calculateNonDiagonalSummary(const Matrix<long double>& matrix) {
    long double summary = 0;
    for (int i = 0; i < matrix.getRows(); ++i) {
        for (int j = i + 1; j < matrix.getRows(); ++j) {
            summary += std::pow(matrix.at(i, j), 2);
        }
    }
    return std::pow(summary, 0.5);
}

```

```

#include "rotating.h"
#include <vector>

int main() {
    Rotating rot(0.0000001);
    Matrix<long double> mat("/home/zloyaloha/programming/Numerical-analysis/lab1/rotating/
test/text.txt");
    auto [eigenvalues, eigenvectors] = rot.solve(mat);

    for (int i = 0; i < eigenvalues.size(); ++i) {
        Matrix<long double> x(mat.getRows(), 1, 0);
        for (int j = 0; j < x.getRows(); ++j) {
            x.at(j, 0) = eigenvectors.at(j, i);
        }

        std::cout << "====Check " << i << "====" << std::endl;

        std::cout << "matrix * eigenvector" << std::endl;
        (mat * x).print();
        std::cout << std::endl;

        std::cout << "eigenvalue * eigenvector" << std::endl;
        (x * eigenvalues[i]).print();
        std::cout << std::endl;
        std::cout << "====" << std::endl;
    }
}

```

Формат входных данных

На рисунке 4.1 приведен пример входных данных.



Рисунок 4.1 – Пример входных данных

На первой строке вводятся два числа n, m – размерность матрицы. На следующих n строках вводятся коэффициенты матрицы. В консоли вводится задаваемая точность.

Формат выходных данных

Ниже вы можете увидеть формат выходных данных.

Result for accuracy = 1e-07 by 7 iterations

eigenvalues

14.1143

5.94541

-12.0597

eigenvectors

0.78338 0.471167 -0.405361

-0.503547 0.863431 0.0304698

0.364357 0.180249 0.913649

=====Check 0=====

matrix * eigenvector

11.0569

-7.10723

5.14266

eigenvalue * eigenvector

11.0569

-7.10723

5.14266

=====

=====Check 1=====

matrix * eigenvector

2.80128

5.13345

1.07165

eigenvalue * eigenvector

2.80128

5.13345

1.07165

=====

=====Check 2=====

matrix * eigenvector

4.88855

-0.367458

-11.0184

eigenvalue * eigenvector

4.88855

-0.367458

-11.0184

=====

Сначала выводится количество итераций, которое потребовалось для достижения желаемой точности, потом выводятся собственные значения, затем собственные векторы. А после выполняется проверка на правильность нахождения СВ и СЗ: матрица умножается на собственный вектор и собственный вектор умножается на соответствующее ему собственное значение.

Заключение

Метод Якоби представляет собой эффективный подход к нахождению собственных значений и векторов, но имеет определенные ограничения в применении. Его основное преимущество - относительная простота реализации для симметричных матриц, где сходимость алгоритма гарантирована математически. Однако вычислительная сложность метода возрастает из-за необходимости многократного расчета углов поворота по

нетривиальным формулам, что может приводить к значительному накоплению вычислительной погрешности в процессе итераций.

Особенно заметно это проявляется при работе с матрицами, имеющими близкие, но не идентичные собственные значения. В таких случаях даже незначительные ошибки округления могут существенно исказить конечный результат. Хотя метод демонстрирует надежную сходимость для симметричных матриц, его практическое применение требует тщательного контроля точности вычислений, особенно при большом числе итераций. Эти особенности делают метод Якоби не всегда оптимальным выбором для задач, требующих высокой точности определения близких собственных значений.

QR-алгоритм

Теория

QR-алгоритм представляет собой мощный инструмент для решения полной проблемы собственных значений произвольных квадратных матриц, включая несимметричные случаи. Суть метода заключается в итеративном приведении исходной матрицы к треугольной или квазитреугольной форме с помощью последовательности ортогональных преобразований.

Основу алгоритма составляет QR-разложение, при котором матрица A представляется в виде произведения ортогональной матрицы Q и верхнетреугольной матрицы R . Для эффективного построения такого разложения часто применяются преобразования Хаусхолдера, определяемые через специально подобранные векторы и матрицы отражения. Эти преобразования последовательно обнуляют поддиагональные элементы столбцов матрицы.

Итерационный процесс QR-алгоритма организуется следующим образом: на каждом шаге выполняется QR-разложение текущей матрицы, после чего полученные Q и R -факторы перемножаются в обратном порядке. В результате такой процедуры матрица постепенно приобретает треугольную форму, причем вещественные собственные значения проявляются на диагонали, а комплексно-сопряженные пары - в виде блоков 2×2 .

Критерием сходимости для вещественных собственных значений служит малость поддиагональных элементов соответствующего столбца. В случае комплексных собственных значений необходимо анализировать диагональные блоки, решая для них характеристические уравнения. Особенностью алгоритма является его универсальность - он позволяет находить как вещественные, так и комплексные собственные значения произвольных квадратных матриц, что делает его одним из наиболее востребованных методов в вычислительной линейной алгебре.

Код программы

```
#include "matrix.h"
#include <cmath>

const double EPS = 1e-10;
const int MAX_ITER = 100;

double norm(const Matrix<double>& v) {
    double sum = 0.0;
```

```

    for (int i = 0; i < v.getRows(); ++i) {
        sum += v.at(i, 0) * v.at(i, 0);
    }
    return std::sqrt(sum);
}

double characteristicRoots(double a, double b, double c, double d, double alpha) {
    double trace = a + d;
    double det = a * d - b * c;

    double discriminant = trace * trace - 4 * det;

    if (discriminant < 0) {
        double realPart = trace / 2;
        double imagPart = sqrt(-discriminant) / 2;
        std::cout << "Корни характеристического уравнения (комплексные):\n";
        std::cout << "\lambda1 = " << realPart << " + " << imagPart << "i\n";
        std::cout << "\lambda2 = " << realPart << " - " << imagPart << "i\n";
        std::cout << "Корень характеристического уравнения (не комплексный):\n";
        std::cout << "\lambda2 = " << alpha << std::endl;
    }
    return discriminant;
}

class QR {
private:
    std::pair<Matrix<double>, Matrix<double>> householder(Matrix<double> A);
    bool isConverged(const Matrix<double>& A, double eps);
public:
    QR() = default;
    void decompose(Matrix<double> A, const double& eps);
};

bool QR::isConverged(const Matrix<double>& A, double eps) {
    int n = A.getCols();

```

```

double sum = 0.0;
for (int i = 1; i < n; ++i)
    for (int j = 0; j < i; ++j)
        sum += std::pow(A.at(i, j), 2);
return sqrt(sum) < eps;
}

std::pair<Matrix<double>, Matrix<double>>> QR::householder(Matrix<double> A)
{
    int n = A.getRows();
    Matrix<double> R = A;
    Matrix<double> Q = Matrix<double>::identity(n);

    for (int k = 0; k < n - 1; ++k) {
        Matrix<double> x(n - k, 1, 0.0);
        for (int i = k; i < n; ++i)
            x.at(i - k, 0) = R.at(i, k);

        Matrix<double> e(n - k, 1, 0.0);
        e.at(0, 0) = 1.0;

        double alpha = norm(x);
        if (x.at(0, 0) >= 0)
            alpha = -alpha;

        Matrix<double> v = x - e * alpha;

        Matrix<double> H_sub = Matrix<double>::identity(n - k) - (v * ~v) * (2.0 / ((~v * v).at(0,
0)));

        Matrix<double> H = Matrix<double>::identity(n);
        for (int i = k; i < n; ++i)
            for (int j = k; j < n; ++j)
                H.at(i, j) = H_sub.at(i - k, j - k);
    }
}

```

```

    R = H * R;
    Q = Q * H;
}

return {Q, R};
}

void QR::decompose(Matrix<double> A, const double& eps) {
    int n = A.getCols();
    int iter = 0;
    while (true) {
        ++iter;
        auto [Q, R] = householder(A);
        A = (R * Q);
        if (isConverged(A, eps)) {
            std::cout << "EIGENVALUES!\n";
            for (int i = 0; i < A.getCols(); ++i) {
                std::cout << A.at(i, i) << '\n';
            }
            return;
        }
        if (std::sqrt(A.at(2, 0) * A.at(2, 0)) < eps && (std::sqrt(A.at(1, 0) * A.at(1, 0)) > eps ||
std::sqrt(A.at(2, 1) * A.at(2, 1)) > eps)) {
            double discr;
            if (std::sqrt(A.at(1, 0) * A.at(1, 0)) > eps) {
                discr = characteristicRoots(A.at(0, 0), A.at(0, 1), A.at(1, 0), A.at(0, 1), A.at(2, 2));
            }
            if (std::sqrt(A.at(2, 1) * A.at(2, 1)) > eps) {
                discr = characteristicRoots(A.at(1, 1), A.at(1, 2), A.at(2, 1), A.at(2, 2), A.at(0, 0));
            }
            if (discr < 0) {
                return;
            }
        }
        if (iter == MAX_ITER) {

```

```

        std::cerr << "Не удалось достичь желаемой точности за " << iter << " итераций\n";
        return;
    }
}

int main() {
    QR qr;
    double eps;
    std::cin >> eps;
    Matrix<double> mat("/home/zloyaloha/Yandex.Disk/MAI/Numerical-analysis/lab1/qr/test/
test2.txt");
    qr.decompose(mat, eps);
}

```

Формат входных данных

На рисунке 5.1 приведен пример входных данных.



```

3 3
1 2 5
-8 0 -6
7 -9 -7

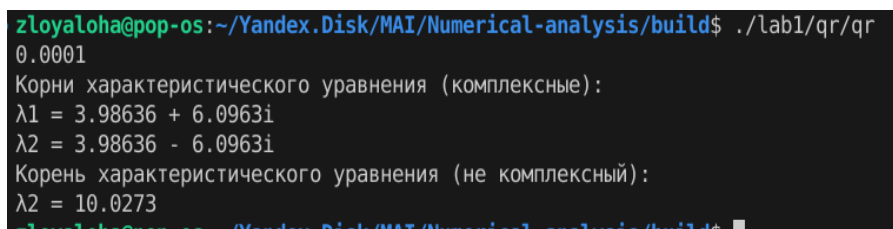
```

Рисунок 5.1 – Пример входных данных

На первой строке вводятся числа n , m – размер матрицы. На следующих n строках вводятся элементы матрицы. Далее через консоль вводится точность ε , с которой должны быть найдены собственные значения.

Формат выходных данных

На рисунке 5.2 приведены выходные данные, являющиеся решением задачи с рисунка 5.1.



```

zloyaloha@pop-os:~/Yandex.Disk/MAI/Numerical-analysis/build$ ./lab1/qr/qr
0.0001
Корни характеристического уравнения (комплексные):
λ1 = 3.98636 + 6.0963i
λ2 = 3.98636 - 6.0963i
Корень характеристического уравнения (не комплексный):
λ2 = 10.0273

```

Рисунок 5.2 – Решение задачи

В выводе присутствует вывод мнимых и действительных СЗ.

Заключение

QR-алгоритм зарекомендовал себя как универсальное и надежное средство для решения задачи нахождения СЗ и СВ. Хотя его базовая реализация обладает значительной вычислительной сложностью, применение специальных приемов (таких как предварительное приведение матрицы к форме Хессенберга) позволяет существенно повысить эффективность метода. Эти оптимизации делают алгоритм практически применимым для решения широкого круга задач, сохраняя при этом его главное преимущество - способность находить все собственные значения и векторы произвольных квадратных матриц. Таким образом, QR-алгоритм остается одним из наиболее востребованных инструментов в вычислительной линейной алгебре, сочетая в себе универсальность и достаточную для практических применений производительность.